

Avant de commencer ces activités, il faut d'abord suivre Activity 5a-Tester-setup-PHP-MySQL.

MySQL est un serveur de bases de données qui se repose sur l'utilisation d'un langage SQL (Structured Query Language). Le serveur comprend un ensemble d'outils et d'interface pour créer des tables, manipuler des données, extraire des données, réaliser des tâches d'administration (création d'utilisateurs, attribution de privilèges, etc.). L'ensemble de ces actions repose sur l'utilisation d'un langage appelé le SQL (Structured Query Language).

Le serveur de bases de données gère lui-même la manière dont sont stockées les données. De nombreux langages comme le PHP ou le Python permettent de se connecter à un serveur MySQL et de générer des requêtes en SQL afin d'accéder aux données d'une base. De nos jours, la plupart des sites Web stockent leurs données dans des bases de données mais également de nombreux logiciels et systèmes (**Firefox** par exemple). MySQL est un des leader dans le monde des serveurs de base de données, il présente l'avantage d'être disponible sous licence GPL.

1. Communication entre PHP et MySQL

1.1 Ce qu'on a fait dans Activity 5a

Dans l'activité 5a, nous avons commencé (notre serveur web / HTTP, installé en cours 4 PHP. Nous avons assuré que Apache (web server) et pour la suite MySQL sont activés dans vos services Docker (ou si vous utilisé UwAmp que le port 8080 ou 8081 à l'école et le porte 3306 est ouverte).

Ensuite on a visité la page de phpMyAdmin <http://localhost:8001/> (à l'école <http://localhost:8080/dashboard/>). Là on a utilisé l'interface web de phpMyAdmin pour créer une BD avec nom "appinfo". On y a ajouté un table (avec nom "table 1" avec 5 champs. Et on a ajouté des entrées dans le table, en utilisant d'abord l'interface web mais aussi un script php (fichier `test-sql.php`).

1.2 Écriture dans une base de données à l'aide de PHP et de formulaire.html

Nous allons enfin créer un site dynamic interessant, qui combiner un front-end (une page web interactive) et notre serveur (BD). D'abord nous allons créer et utiliser un formulaire html pour ajouter des entrées à notre table, comme l'image en bas. Je vous suggère de faire aussi l'activité optionnel sur les formulaires (section PHP, Activity-4b-PHP-FormulairesHTML.pdf).

Nom:

Prénom :

Sexe : ☐ Masculin ☐ Féminin

Année de naissance :

Comment faire. Dans le répertoire `www/` du serveur HTTP, créer le fichier `formulaire.html` tel que :

```
<!DOCTYPE html>
<html>
  <head>
    <title>Formulaire base donnée</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <form method="get" action="traitement.php">
      Enregistrement d'un utilisateur<br />
      Nom<input type="text" name="fnom" /><br />
      Prénom<input type="text" name="fpre" /><br />
      Sexe<br />
      Masculin: <input type="radio" name="fsex" value="M" /><br />
      Féminin: <input type="radio" name="fsex" value="F" /><br />
      Année de naissance<br />
      <select name="fannee">
        <option value="1997">1997</option>
        <option value="1996">1996</option>
        <option value="1995">1995</option>
        <option value="1994">1994</option>
        <option value="1993">1993</option>
        <option value="1992">1992</option>
        <option value="1991">1991</option>
        <option value="1990">1990</option>
      </select>

      <input type="submit" value="Envoyer" />

    </form>
  </body>
</html>
```

Ce formulaire appelle un fichier `traitement.php` dont le code est le suivant :

```
<?php

/* recuperation donnees formulaire */
$nom= $_GET["fnom"];
$prenom= $_GET["fpre"];
$sexe= $_GET["fsex"];
$annee= $_GET["fannee"];

/* identifiants pour la connexion avec la BDD */
/* à adapter selon la configuration */
$host='db'; // Docker, avec UwAmp plutôt "127.0.0.1:3306"
$user="php_docker"; // Docker, avec UwAmp plutôt "root" ou rien
$pass="password"; // Docker, avec UwAmp plutôt "test" ou rien
$base="appinfo";

/* connexion à la BDD */
$bdd = mysqli_connect($host,$user,$pass,$base);
if (!$bdd)
  die('Echec de connexion au serveur de base de
```

```
données:'.mysqli_connect_error().' '.mysqli_connect_errno());
echo "connecté à la base de données <br>";
echo "J'ai ajouté ".$nom.$prenom.$sexe.$annee." dans la BD";

/* Fixe le jeu de caractères de la BDD en UTF-8 */
mysqli_set_charset($bdd, "utf8");

/* envoi de la requete d'ecriture */
$query="INSERT INTO table1 (nom,prenom,sexe,annee)
VALUES( '$nom', '$prenom', '$sexe', '$annee' )";
mysqli_query($bdd,$query);

/* deconnexion de la BDD */
mysqli_close($bdd);
?>
```

Q1 :

Tester le formulaire et vérifier que les données sont bien présentes dans la base de données (il faudra modifier le fichier PHP en fonction de la configuration de base, i.e., mot de passe, nom de la base...). Regarder les logs si le comportement semble bizarre.

Q2 :

Quelle ligne permet de se connecter à la base de données ?

Q3 :

Quel est le rôle de la requête :

```
INSERT INTO table1 (nom,prenom,sexe,annee)
VALUES( '$nom', '$prenom', '$sexe', '$annee' ) ?
```

1.3 Lecture d'une base de données à l'aide de PHP

Pour lire et afficher le contenu de la table dans une page Web, nous allons utiliser le fichier `lecture.php` suivant qu'il faudra adapter à votre configuration :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Lecture de la table table1</title>
  </head>
  <body>

  <?php
    /* identifiants pour la connexion avec la BDD */
    /* à adapter selon la configuration */
    $host='db'; // Docker, avec UwAmp plutôt "127.0.0.1:3306"
    $user="php_docker"; // Docker, avec UwAmp plutôt "root" ou rien
    $pass="password"; // Docker, avec UwAmp plutôt "test" ou rien
    $base="appinfo";

    /* connexion à la BDD */
```

```
$bdd = mysqli_connect($host,$user,$pass,$base);

if (!$bdd)
    die('Echec de connexion au serveur de base de
donnees:'.mysqli_connect_error().' '.mysqli_connect_errno());

echo "connexion réussie à la base de données... <br/>";

/* fixe le jeu de caractères de la BDD en UTF-8 */
mysqli_set_charset($bdd, "utf8");

/* envoi de la requête de lecture */
$query="SELECT * FROM table1 LIMIT 0,30"; // requete SQL

if ($reponse=mysqli_query($bdd,$query)) {
    $nb=mysqli_num_rows($reponse); // nb de lignes du résultat
    echo "Nombre de personnes dans la base:".$nb."<br/><br/>";

    if ($nb!=0) {
        while($ligne = mysqli_fetch_array($reponse)) {
            $nom = $ligne['nom'];
            $prenom = $ligne['prenom'];
            echo "Nom: ".$nom." Prénom: ".$prenom."<br/>";
        }
    }

    /* libération la mémoire utilisée par les réponses */
    mysqli_free_result($reponse);
}

/* déconnexion de la BDD */
mysqli_close($bdd);
?>

</body>
</html>
```

Q4 :
Faire pointer **Firefox** sur ce script et vérifier que l'affichage est correct. A l'aide de la fonction `mysqli_error()`, ajouter un contrôle de l'existence de la table.

Q5 :
Quel est le rôle de la ligne : `$query="SELECT * FROM ... LIMIT 0,30";` ?

Q6 :
Quel est le rôle de la ligne : `while($ligne = mysqli_fetch_array($reponse))` ?

Q7 :
Modifier le fichier afin d'afficher tous les champs de la table `table1` dans un tableau HTML. Tester.

2. Application 1

L'ensemble des sites demandant une connexion (Webmail, forums, etc.) utilisent une base de données pour stocker les informations relatives à l'utilisateur : login, mot de passe, nom, prénom, date de dernière visite, etc.

Accès aux données

Indentifiant:
 Mot de passe:

 Nouveau? Veuillez vous inscrire [Ici](#).

On se propose de réaliser l'accès d'un tel site comme montré ci- contre. L'utilisateur entre son identifiant et son mot de passe et s'ils sont corrects, il accède à une page du type donné ci- dessous :

Nous sommes le 2020-10-20 15:59:48.

Bonjour Jean Dupont

Votre dernier passage remonte au 2020-10-20 15:52:26

[Retour à l'accueil](#)

La page donne la date actuelle ainsi que celle du précédent passage de l'utilisateur. Le nom et le prénom de l'utilisateur sont utilisé dans une formule de bienvenue.

Comme indiqué sur la première image (page d'accès), il sera possible de s'inscrire grâce à un formulaire :

Formulaire d'inscription

Nom:
 Prénom:
 Indentifiant:
 Mot de passe:

[Retour](#)

Q8 :

Réaliser ce site, et le tester pour au moins 5 utilisateurs inscrits.

Q9 :
L'administrateur (admin) doit pouvoir se connecter et afficher la liste des inscrits avec leurs informations mais sans les mots de passe.

Q10 :
Modifier la page de l'administrateur avec la possibilit  de r initialiser le mot de passe d'un utilisateur   son pr nom   l'envers.

3. Application 2

Dans cette partie, nous allons am liorer le site pr c dent.

Q11 :
Rajouter, lors de l'inscription, l'ajout d'une photo   son profil. Il faudra limiter la taille du fichier   100ko. Lors de sa connexion, la page d'accueil de l'utilisateur devra afficher sa photo. En cas d'absence de photo, une photo par d faut sera affich e en fonction du sexe de la personne. Pour faire les tests, vous r cup rez les photos de personnages importants dans le domaine de l'informatique et informatique industrielle (Annie Easley, Richard Stallman, Margaret Hamilton, Tim Berners-Lee ou encore Linus Torvalds, leur page Wikipedia fournira la photo n cessaire).

Q12 :
Lors de son inscription, l'utilisateur devra mentionner sa ville de r sidence. L'objectif est d'afficher la m t o   l'utilisateur une fois qu'il est connect .   cet effet, vous utiliserez curl et le site `wttr.in` :

Cf <https://github.com/chubin/wttr.in>

Cf <http://wttr.in/Orsay>

Cf <http://wttr.in/Orsay?0>

Cf <https://www.php.net/manual/en/curl.examples.php>

4. Annexes

Une table d'une base de données MySQL peut contenir différents types de données ; des caractères, des valeurs numériques, des dates et heures, des données binaires et même des listes.

Ci-dessous sont présentés les types les plus couramment rencontrés.

Caractères

Type	Description	
CHAR	Chaîne fixe de n caractères ou octets	Taille fixe (maximum 255 caractères)
VARCHAR	Chaîne variable de n caractères ou octets	Taille variable (maximum de 65535 caractères)
BINARY	Chaîne fixe de n octets	Taille fixe (maximum 255 octets)
TINYTEXT	Flot de n octets	Taille fixe (maximum 255 octets)
TEXT	Flot de n octets	Taille fixe (maximum 65535 octets)
MEDIUMTEXT	Flot de n octets	Taille fixe (maximum 16 mégaoctets)

Nombres

Type	Description	
BIT	Ensemble de n bits. Taille de 1 à 64 (par défaut 1)	
TINYINT	Entier (sur 1 octet) de -128 à +127 signé, de 0 à 255 non signé (Attribut UNSIGNED)	
SMALLINT	Entier (sur 2 octets) de -32768 à +32767 signé, de 0 à 65535 non signé (Attribut UNSIGNED)	
INT	Entier (sur 4 octets) de -2 147 483 648 à +2 147 483 647 signé, de 0 à 4 294 967 295 non signé (Attribut UNSIGNED)	
DECIMAL	Décimal à virgule fixe (maximum 30 chiffres après la virgule)	
FLOAT	Flottant (de 4 à 8 octets) en précision simple (jusqu'à 7 décimales)	

Dates et heures

Type	Description	
DATE	Date	Sur 3 octets, format : 'YYYY-MM-DD'
DATETIME	Date et heure	Sur 8 octets, format : 'YYYY-MM-DD HH:MM:SS'
TIME	heure	Sur 3 octets, format : 'HHH-MM-SS'

Objets binaires (stockage d'images, sons, vidéos, etc.)

Type	Description	
TINYBLOB	Flots de n octets	Taille fixe (maximum 255 octets)
BLOB	Flots de n octets	Taille fixe (maximum 65535 octets)
MEDIUMBLOB	Flots de n octets	Taille fixe (maximum 16 mégaoctets)
LOB	Flots de n octets	Taille fixe (maximum 4,29 gigaoctets)